



BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

MÃO ROBÓTICA CONTROLADA POR VISÃO COMPUTACIONAL

Documentação completa para desenvolvimento de um projeto similar ao vídeo de referência

Aluna	Ana Clara Ortiz Ochner
Instituição	UniFAJ - Centro Universitário de Jaguariúna
Área	Robótica • IoT • Visão Computacional
Ano	2026

Jaguariúna - SP

SUMÁRIO DA DOCUMENTAÇÃO

Este material foi estruturado como uma apostila técnica para orientar a construção, os testes e a apresentação acadêmica do protótipo.

1. Escopo e objetivo do projeto
 2. Arquitetura e funcionamento
 3. Materiais e componentes
 4. Estrutura mecânica e impressão 3D
 5. Sistema de tendões
 6. Eletrônica e alimentação
 7. Montagem passo a passo
 8. Ambiente de software
 9. Visão computacional com MediaPipe
 10. Comunicação serial e firmware ESP32
 11. Calibração
 12. Plano de testes
 13. Diagnóstico de falhas
 14. Melhorias e evolução
 15. Cronograma de execução
 16. Segurança e limitações
 17. Checklist final
 18. Referências técnicas
- Apêndice A. Código Python base
- Apêndice B. Firmware ESP32 base

Referência principal: vídeo fornecido pelo projeto: https://www.youtube.com/watch?v=_GSqwtdkvcs. Os componentes confirmados pela referência são apresentados separadamente das recomendações de implementação.

Como utilizar esta documentação

1. Leia primeiro as seções de arquitetura, alimentação e segurança.
2. Monte e teste apenas um servo antes de conectar a estrutura mecânica.
3. Valide um único dedo com tendão e retorno elástico.
4. Repita o mecanismo nos cinco dedos somente após a primeira validação.
5. Integre a visão computacional por último, quando a mão já responder corretamente aos comandos manuais.

1. ESCOPO E OBJETIVO DO PROJETO

O projeto consiste no desenvolvimento de uma mão robótica articulada capaz de reproduzir, aproximadamente em tempo real, a flexão dos dedos de uma mão humana posicionada diante de uma câmera. A webcam captura a mão; um software em Python identifica os pontos anatômicos; os valores de flexão são enviados ao ESP32; e os servomotores movimentam os dedos por meio de tendões artificiais.

Escopo acadêmico O protótipo deve ser tratado como sistema robótico educacional. Não deve ser utilizado como dispositivo médico ou prótese de uso humano sem projeto específico, validação de segurança e requisitos regulatórios.

1.1 Objetivo geral

Construir um protótipo funcional que integre impressão 3D, eletrônica, sistemas embarcados e visão computacional para reproduzir movimentos básicos dos dedos.

1.2 Objetivos específicos

- Desenvolver uma estrutura mecânica articulada para cinco dedos.
- Acionar cada dedo de forma independente.
- Reconhecer a mão humana por webcam.
- Calcular o grau de flexão dos dedos.
- Transmitir cinco posições ao microcontrolador.
- Controlar os servomotores por PWM.
- Implementar limites de segurança e suavização de movimentos.
- Avaliar latência, estabilidade e capacidade de preensão.

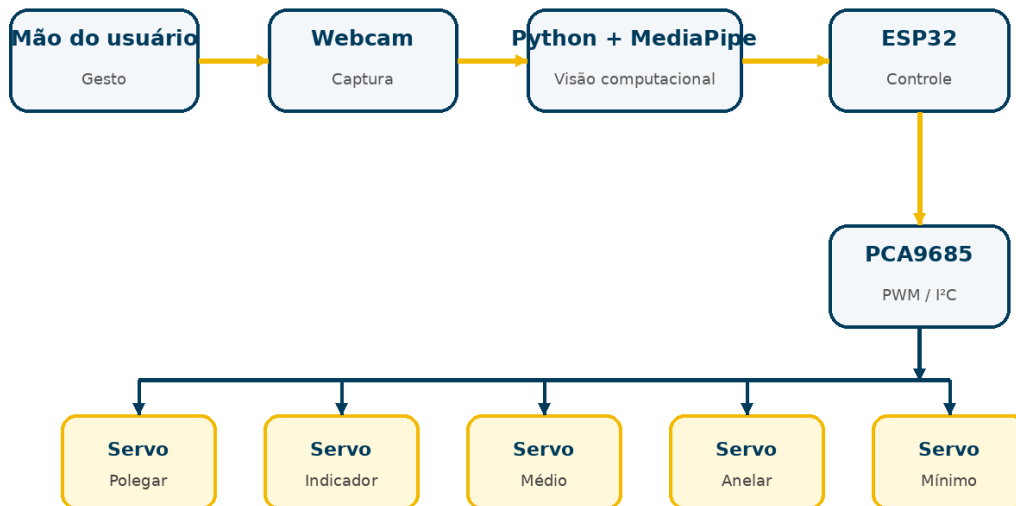
1.3 O que foi confirmado na referência

Item	Situação	Uso no projeto
Python + MediaPipe	Confirmado	Processamento da mão e interpretação dos dedos
ESP32	Confirmado	Microcontrolador responsável pelo comando dos atuadores
PCA9685	Confirmado	Driver PWM para múltiplos servomotores
Servomotores	Confirmado	Atuação mecânica dos dedos
Modelo exato dos servos	Não identificável	Utilizar equivalente adequado ao torque necessário
Fonte e corrente exatas	Não identificáveis	Dimensionar de acordo com os servos escolhidos
STL / dimensões mecânicas	Não confirmadas	Adotar modelo de mão articulada compatível

2. ARQUITETURA E FUNCIONAMENTO

O sistema é dividido em quatro camadas: captura, interpretação, controle e atuação. Essa separação facilita os testes e permite substituir componentes sem refazer todo o projeto.

ARQUITETURA DO SISTEMA



Fluxo: gesto → visão computacional → controle → atuação mecânica

2.1 Fluxo de dados

1. A webcam captura quadros de vídeo.
2. O MediaPipe localiza os landmarks da mão.
3. O programa calcula a flexão de cada dedo.
4. Os valores são filtrados para reduzir tremores.
5. Cinco ângulos são enviados pela porta serial.
6. O ESP32 recebe e valida o pacote.
7. O PCA9685 gera os sinais PWM.
8. Os servos puxam os tendões e flexionam os dedos.

2.2 Estratégia modular recomendada

Regra de ouro não monte a mão completa de uma vez. Primeiro valide ESP32 + PCA9685 + 1 servo; depois 1 dedo; depois 5 servos; por fim integre a visão computacional.

3. MATERIAIS E COMPONENTES

3.1 Eletrônica principal

Qtd.	Componente	Função	Observação
1	ESP32 DevKit	Controle principal	Possui comunicação USB e barramento I ² C
1	PCA9685 16 canais	Geração de PWM	Permite controlar vários servos pelo I ² C
5	Microservos	Movimento dos dedos	MG90S recomendado como ponto de partida
1	Fonte regulada 5 V / 5 A	Alimentação dos servos	Ajustar conforme corrente real dos atuadores
1	Capacitor eletrolítico ~1000 µF	Estabilização	Ajuda a reduzir quedas rápidas de tensão
1	Cabo USB	PC ↔ ESP32	Programação e serial
Vários	Jumpers / fios	Interligações	Preferir conexões firmes
1	Chave liga/desliga	Segurança	Interrompe alimentação dos atuadores
1	Fusível adequado	Proteção	Dimensionar segundo a fonte e o circuito

3.2 Parte mecânica

- Palma e dedos articulados impressos em 3D.
- Antebraço ou base para fixar os servomotores.
- PLA para protótipo inicial ou PETG para maior resistência.
- Fio de nylon/linha de pesca para tendões.
- Fio elástico ou pequenas molas para retorno.
- Eixos, pinos ou parafusos M2/M3 nas articulações.
- Horns dos servos e parafusos de fixação.
- Abraçadeiras, termorretrátil e organizadores de cabo.

3.3 Software e ferramentas

Categoria	Itens
Computador	Notebook/PC com webcam ou webcam USB
Python	Python 3, OpenCV, MediaPipe, NumPy e PySerial
ESP32	Arduino IDE ou PlatformIO
Bibliotecas	Wire + Adafruit PWM Servo Driver
Modelagem / impressão	Software de fatiamento e arquivos STL da mão escolhida
Medição	Multímetro; opcionalmente fonte de bancada e sensor de corrente

Escolha dos servos SG90 pode ser suficiente para testes leves; MG90S oferece engrenagem metálica em formato compacto; MG996R exige mais espaço e corrente. O modelo ideal depende da geometria da mão e da força desejada.

4. ESTRUTURA MECÂNICA E IMPRESSÃO 3D

A mecânica deve permitir que cada dedo flexione com baixo atrito e retorne à posição aberta quando o tendão for liberado. A geometria das articulações é tão importante quanto o código: um dedo com atrito excessivo fará o servo trabalhar no limite e poderá causar aquecimento.

4.1 Divisão recomendada das peças

- Palma principal.
- Polegar independente.
- Indicador, médio, anelar e mínimo.
- Falanges com passagens internas para o tendão.
- Base/antebraço para servos, ESP32 e PCA9685.
- Tampa removível para manutenção da eletrônica.

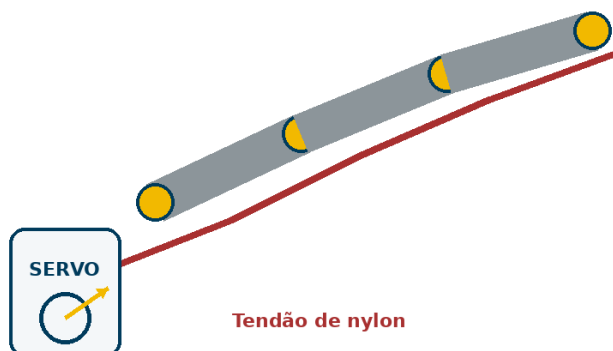
4.2 Parâmetros iniciais de impressão

Parâmetro	Sugestão inicial
Material	PLA para protótipo; PETG se precisar de maior resistência
Preenchimento	15% a 25%
Paredes	3 ou mais nas articulações e pontos de carga
Altura de camada	0,20 mm como referência
Teste inicial	Imprimir somente um dedo antes da mão completa

Melhoria de projeto preveja canais suaves para o nylon e uma tampa de manutenção quase invisível no antebraço. Isso melhora o acabamento sem dificultar o acesso aos servos e à eletrônica.

5. SISTEMA DE TENDÕES

PRINCÍPIO MECÂNICO DO TENDÃO



1. Servo puxa o tendão

2. O fio transmite a força
3. As falanges flexionam
4. Elástico ou mola retorna o dedo

Recomendação

Teste primeiro apenas 1 dedo + 1 servo.
Depois replique para os outros dedos.

Cada servo atua como um “músculo”. Ao girar, o horn enrola ou puxa o fio de nylon. O tendão transmite a força pelas falanges e causa a flexão. Para a extensão, um elástico ou mola devolve o dedo à posição aberta.

5.1 Roteamento do tendão

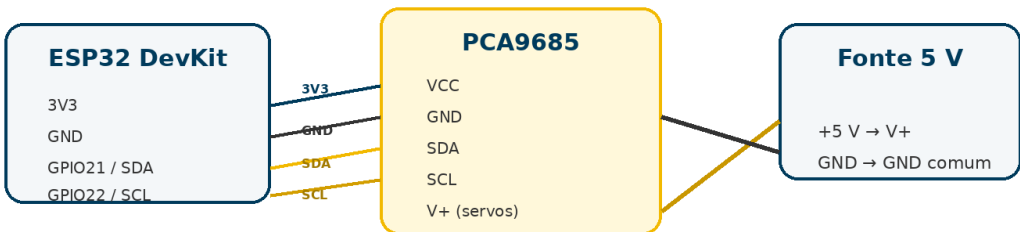
1. Prenda o nylon próximo à ponta do dedo.
2. Passe o fio pelos canais das falanges sem criar arestas cortantes.
3. Conduza o fio pela palma até o antebraço.
4. Direcione cada tendão ao servo correspondente.
5. Com o servo centralizado, deixe o dedo aberto e faça a fixação no horn.
6. Ajuste a pré-tensão: firme o suficiente para responder, mas sem puxar o dedo em repouso.

5.2 Retorno do dedo

- Elástico: barato e fácil, porém perde tensão com o tempo.
- Mola: retorno mais previsível, mas exige projeto mecânico adequado.
- Segundo tendão: permite flexão e extensão ativas, porém duplica atuadores ou complexidade.

6. ELETRÔNICA E ALIMENTAÇÃO

DIAGRAMA DE LIGAÇÃO ELÉTRICA



Canais PWM



Todos os GNDs devem ser comuns.
Não alimente os 5 servos pela USB do computador.
Use fonte externa dimensionada para a corrente dos servos.

6.1 ESP32 ↔ PCA9685

ESP32	PCA9685	Função
3V3	VCC	Alimentação lógica
GND	GND	Referência comum
GPIO21	SDA	Dados I ² C
GPIO22	SCL	Clock I ² C

6.2 Servos ↔ PCA9685

Canal	Atuador
0	Polegar
1	Indicador
2	Médio
3	Anelar
4	Mínimo

ATENÇÃO À ALIMENTAÇÃO Não utilize a USB do computador ou o pino de 5 V do microcontrolador para alimentar simultaneamente cinco servos. Utilize fonte externa dedicada aos atuadores e mantenha o GND comum entre fonte, PCA9685 e ESP32.

6.3 Sequência segura de energização

1. Confirme a polaridade da fonte com multímetro.
2. Conecte apenas ESP32 + PCA9685 sem servos.
3. Execute um scanner I²C e confirme o endereço do PCA9685.
4. Conecte um servo e teste posição neutra.
5. Desligue a fonte antes de conectar servos adicionais.
6. Conecte os cinco servos apenas após os testes individuais.

7. MONTAGEM PASSO A PASSO

Fase 1 - Bancada eletrônica mínima

- ☐ Montar ESP32 e PCA9685.
- ☐ Conectar VCC, GND, SDA e SCL.
- ☐ Alimentar o PCA9685 corretamente.
- ☐ Carregar programa de teste.
- ☐ Confirmar um servo girando para posições conhecidas.

Fase 2 - Validação mecânica de um dedo

- ☐ Imprimir um dedo.
- ☐ Montar pinos/eixos das falanges.
- ☐ Passar o tendão.
- ☐ Instalar retorno elástico.
- ☐ Fixar um servo na base.
- ☐ Centralizar o servo em 90° antes de prender o horn.
- ☐ Prender tendão com o dedo aberto.
- ☐ Testar abertura e fechamento progressivos.

Fase 3 - Replicação para os cinco dedos

- ☐ Imprimir palma e os outros dedos.
- ☐ Instalar todos os tendões.
- ☐ Fixar cinco servos no antebraço.
- ☐ Identificar fisicamente cada conector.
- ☐ Executar teste de um canal por vez.
- ☐ Definir limites individuais de cada dedo.

Fase 4 - Integração com o computador

- ☐ Instalar ambiente Python.
- ☐ Testar webcam.
- ☐ Executar detecção da mão.
- ☐ Verificar landmarks na tela.
- ☐ Calcular flexão individual.
- ☐ Abrir comunicação serial.
- ☐ Enviar ângulos sem a mecânica conectada.
- ☐ Conectar tendões somente depois de validar os comandos.

Critério para avançar só passe para a fase seguinte quando a anterior funcionar de forma repetível. Isso facilita localizar falhas e evita quebrar a estrutura mecânica por um erro de software.

8. AMBIENTE DE SOFTWARE

8.1 Estrutura de pastas sugerida

```
robotic-hand/  
├── firmware/  
│   └── esp32_hand.ino  
├── vision/  
│   ├── main.py  
│   └── hand_landmarker.task  
├── mechanical/  
├── schematics/  
├── tests/  
└── documentation/
```

8.2 Ambiente Python

```
python -m venv .venv  
  
# ativar o ambiente virtual  
# Windows:  
.venv\Scripts\activate  
  
# instalar dependências  
pip install mediapipe opencv-python numpy pyserial
```

8.3 Firmware

- Instalar o suporte à placa ESP32 na Arduino IDE.
- Instalar a biblioteca Adafruit PWM Servo Driver.
- Selecionar a porta serial correta.
- Utilizar 115200 baud para facilitar a comunicação com o Python.

9. VISÃO COMPUTACIONAL COM MEDIAPIPE

O modelo de mãos fornece 21 landmarks. Para o projeto, esses pontos são usados para estimar a flexão de cada dedo. A abordagem por ângulo tende a ser mais robusta do que simplesmente comparar coordenadas verticais, pois funciona melhor com diferentes posições da mão.

9.1 Landmarks úteis

Dedo	Pontos de referência	Exemplo de cálculo
Polegar	2, 3, 4	ângulo em torno da articulação 3
Indicador	5, 6, 8	MCP → PIP → TIP
Médio	9, 10, 12	MCP → PIP → TIP
Anelar	13, 14, 16	MCP → PIP → TIP
Mínimo	17, 18, 20	MCP → PIP → TIP

9.2 Conversão para o servo

```
angulo_mao = limitar(angulo_mao, mao_fechada, mao_aberta)
normalizado = (angulo_mao - mao_fechada) / (mao_aberta - mao_fechada)
angulo_servo = servo_fechado - normalizado * (servo_fechado - servo_aberto)
```

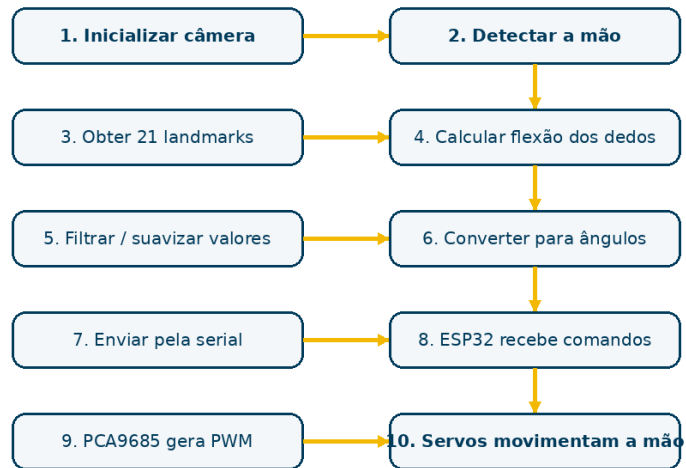
9.3 Suavização

A detecção visual oscila alguns pixels entre quadros. Para evitar tremores nos motores, aplique filtro exponencial e deadband.

```
filtrado = ALPHA * atual + (1 - ALPHA) * anterior

# deadband
if abs(novo - atual_servo) > 2:
    enviar(novo)
```

FLUXO DE EXECUÇÃO DO SOFTWARE



Resultado: a mão robótica reproduz a flexão observada pela webcam em tempo quase real.

10. COMUNICAÇÃO SERIAL E FIRMWARE

Uma estratégia simples é enviar uma linha CSV contendo cinco posições. Exemplo:

```
35, 120, 145, 140, 130
```

A ordem deve ser fixa: polegar, indicador, médio, anelar e mínimo. O ESP32 lê a linha, valida cinco números e atualiza os canais do PCA9685.

10.1 Formato de protocolo recomendado

Campo	Descrição
Thumb	0 a 180 ou faixa calibrada
Index	0 a 180 ou faixa calibrada
Middle	0 a 180 ou faixa calibrada
Ring	0 a 180 ou faixa calibrada
Pinky	0 a 180 ou faixa calibrada
Finalizador	caractere de nova linha \n

10.2 Robustez adicional

- Descartar linhas incompletas.
- Aplicar constrain() para nunca exceder limites seguros.
- Usar timeout: se o PC parar de enviar, manter posição neutra ou liberar a mão.
- Adicionar caractere de início/fim ou checksum em uma versão avançada.

11. CALIBRAÇÃO

A calibração deve ser feita separadamente para cada dedo porque comprimento do tendão, posição do horn e geometria das articulações variam.

11.1 Centralização inicial

1. Retire ou afrouxe o tendão.
2. Comande o servo para 90°.
3. Instale o horn aproximadamente perpendicular ao corpo do servo.
4. Posicione o dedo totalmente aberto.
5. Prenda o tendão com baixa pré-tensão.

11.2 Busca dos limites seguros

Avance em passos pequenos, por exemplo 5° ou 10°. Pare imediatamente se houver travamento, ruído excessivo, deformação do dedo ou aumento abrupto de corrente.

Dedo	Aberto (exemplo)	Fechado (exemplo)	Valor real
Polegar	30°	135°	preencher após teste
Indicador	25°	145°	preencher após teste
Médio	25°	150°	preencher após teste
Anelar	30°	145°	preencher após teste
Mínimo	35°	140°	preencher após teste

Importante os valores acima são exemplos de documentação, não limites universais. A geometria real da sua impressão determina os ângulos seguros.

12. PLANO DE TESTES

12.1 Testes unitários

Teste	Procedimento	Critério de aprovação
I ² C	Executar scanner com PCA9685 conectado	Endereço detectado de forma estável
Servo único	Comandar 30°, 60°, 90°, 120°	Movimento sem reinicialização do ESP32
Cada canal	Ativar CH0 a CH4 individualmente	Somente o servo esperado deve se mover
Tensão	Abrir/fechar um dedo 20 vezes	Sem enrosco ou perda significativa de tensão
Webcam	Detectar mão durante 1 minuto	Detecção estável e sem travamentos
Serial	Enviar pacotes de teste	ESP32 recebe cinco valores corretamente

12.2 Testes de integração

- Mão totalmente aberta.
- Punho fechado.
- Indicador isolado.
- Sinal de paz.
- Flexão parcial de cada dedo.
- Movimentos rápidos e lentos.
- Perda temporária da mão na imagem.
- Desconexão do cabo serial.

12.3 Métricas para apresentação

Métrica	Como medir
Latência	tempo entre gesto e resposta visível do robô
FPS	quadros processados por segundo
Estabilidade	variação do servo com a mão parada
Precisão	quantos gestos são reproduzidos corretamente
Consumo	corrente total dos servos
Força / preensão	objetos leves que o protótipo consegue segurar
Temperatura	aquecimento dos servos em uso contínuo

13. DIAGNÓSTICO DE FALHAS

Sintoma	Causas prováveis	Ação recomendada
ESP32 reinicia ao mover servos	Fonte insuficiente / queda de tensão	Usar fonte externa adequada, GND

		comum e capacitor
Servos tremem	Ruído, detecção instável ou fonte fraca	Aplicar filtro, deadband e revisar alimentação
Um dedo fecha ao contrário	Servo ou mapeamento invertido	Inverter faixa no software ou parâmetro do servo
Dedo não abre completamente	Retorno fraco / tendão preso	Revisar elástico, atrito e roteamento
Servo aquece	Mecanismo travando ou limite excessivo	Reduzir ângulo máximo e atrito
PCA9685 não é detectado	SDA/SCL/GND incorretos	Verificar conexões, endereço e alimentação lógica
Movimento atrasado	Processamento / serial lento	Reduzir resolução, otimizar loop e frequência de envio
Mão some da câmera	Iluminação ou enquadramento	Melhorar luz e manter a mão visível

Diagnóstico eficiente sempre separe problema elétrico, mecânico e de software. Desconecte os tendões para saber se o travamento vem do servo/circuito ou da estrutura impressa.

14. MELHORIAS E EVOLUÇÃO

14.1 Sensores nas pontas dos dedos

Sensores FSR podem indicar contato e pressão aproximada. O firmware pode parar ou reduzir a velocidade quando a força atingir um limite, protegendo o objeto e o mecanismo.

14.2 Monitoramento de corrente

Um sensor como INA219 permite registrar consumo e detectar travamento. Um pico de corrente combinado com pouca variação de posição é um bom indicador de esforço excessivo.

14.3 Polegar com dois graus de liberdade

Um segundo servo no polegar permite separar flexão e oposição, aumentando bastante a qualidade da preensão.

14.4 Dashboard

- Posição atual dos cinco dedos.
- FPS da visão computacional.
- Latência estimada.
- Estado da conexão serial.
- Corrente dos atuadores.
- Indicador de emergência e falhas.

14.5 Evolução por níveis

Nível	Recursos
1	Cinco dedos + visão computacional
2	Suavização + gestos + calibração individual
3	Sensores de pressão
4	Dashboard + telemetria + corrente
5	Controle adaptativo e reconhecimento de objetos

15. CRONOGRAMA DE EXECUÇÃO

Semana	Foco	Entrega
1	Mecânica	Um dedo impresso, articulado e com passagem do tendão
2	Eletrônica	ESP32 + PCA9685 + 1 servo funcionando
3	Mão completa	Cinco servos e cinco dedos montados
4	Visão computacional	Landmarks da mão detectados pela webcam
5	Integração	Python enviando posições ao ESP32
6	Calibração	Limites, suavização e estabilidade
7	Melhorias	Sensores, dashboard ou acabamento
8	Apresentação	Métricas, vídeo, documentação e demonstração

16. SEGURANÇA E LIMITAÇÕES

- Desligar a alimentação antes de alterar cabos dos servos.
- Evitar curto-circuito na fonte de alta corrente.
- Adicionar fusível e chave liga/desliga.
- Não forçar manualmente o servo energizado.
- Programar limites individuais de software.
- Manter dedos humanos afastados das articulações durante testes de força.
- Parar o teste se houver cheiro, aquecimento anormal ou ruído de engrenagem.
- Utilizar o projeto somente como protótipo acadêmico/robótico.

Parada de emergência em uma versão mais robusta, utilize um botão que desligue a alimentação dos servos ou atue sobre o pino OE do PCA9685 para desabilitar as saídas rapidamente.

16.1 Limitação em relação a uma prótese real

Uma prótese destinada a uma pessoa exige requisitos adicionais de ergonomia, segurança funcional, materiais, biocompatibilidade, controle de força, confiabilidade, avaliação clínica e regulamentação. Este projeto não substitui esse processo.

17. CHECKLIST FINAL DO PROJETO

Mecânica

- ☐ Articulações giram sem travar.
- ☐ Tendões não raspam em arestas cortantes.
- ☐ Retorno elástico funciona nos cinco dedos.
- ☐ Servos estão firmemente fixados.
- ☐ Existe acesso para manutenção interna.

Eletrônica

- ☐ GND comum confirmado.
- ☐ Fonte externa dos servos dimensionada.
- ☐ PCA9685 detectado pelo ESP32.
- ☐ Cinco canais identificados.
- ☐ Chave geral / emergência disponível.
- ☐ Cabos organizados e sem partes expostas perigosas.

Software

- ☐ Webcam inicia corretamente.
- ☐ Mão é detectada de forma consistente.
- ☐ Cinco valores são calculados.
- ☐ Filtro reduz tremores.
- ☐ Serial envia e recebe sem erros.
- ☐ Limites individuais são aplicados.
- ☐ Timeout ou estado seguro existe quando a mão não é detectada.

Demonstração

- ☐ Mão aberta.
- ☐ Punho fechado.
- ☐ Indicador.
- ☐ Sinal de paz.
- ☐ Flexão parcial.
- ☐ Preensão de objeto leve.
- ☐ Métricas registradas.
- ☐ Vídeo de backup preparado para apresentação.

18. REFERÊNCIAS TÉCNICAS

As referências abaixo sustentam a arquitetura e as escolhas de implementação. O vídeo fornecido é a referência visual principal; as documentações oficiais são utilizadas para os detalhes de software e eletrônica.

Vídeo de referência — YouTube - https://www.youtube.com/watch?v=_GSqwtdkvcs

Google MediaPipe - Hand Landmarker — Documentação oficial do Google AI Edge para detecção de mãos e landmarks.

Adafruit PCA9685 16-Channel PWM / Servo Driver — Documentação de ligação, alimentação e controle dos canais PWM.

Espressif Arduino-ESP32 I²C — Documentação oficial do barramento I²C no ESP32.

Adafruit PWM Servo Driver Library — Biblioteca utilizada no firmware de exemplo.

APÊNDICE A - CÓDIGO PYTHON BASE

O exemplo abaixo implementa captura de vídeo, MediaPipe, cálculo de ângulos, filtro e comunicação serial. Ajuste COM5, o caminho do modelo e os limites conforme o seu computador e a sua mão robótica.

```
import cv2
import time
import serial
import numpy as np
import mediapipe as mp

SERIAL_PORT = "COM5"
BAUD_RATE = 115200
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.02)

BaseOptions = mp.tasks.BaseOptions
HandLandmarker = mp.tasks.vision.HandLandmarker
HandLandmarkerOptions = mp.tasks.vision.HandLandmarkerOptions
RunningMode = mp.tasks.vision.RunningMode

options = HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path="hand_landmarker.task"),
    running_mode=RunningMode.VIDEO,
    num_hands=1,
    min_hand_detection_confidence=0.5,
    min_hand_presence_confidence=0.5,
    min_tracking_confidence=0.5
)
landmarker = HandLandmarker.create_from_options(options)

def calc_angle(a, b, c):
    a = np.array([a.x, a.y])
    b = np.array([b.x, b.y])
    c = np.array([c.x, c.y])
    ba = a - b
    bc = c - b
    cos_angle = np.dot(ba, bc) / (np.linalg.norm(ba) * np.linalg.norm(bc) + 1e-8)
    return np.degrees(np.arccos(np.clip(cos_angle, -1.0, 1.0)))

def angle_to_servo(angle, hand_closed=70, hand_open=170,
                  servo_open=10, servo_closed=160):
    angle = np.clip(angle, hand_closed, hand_open)
    normalized = (angle - hand_closed) / (hand_open - hand_closed)
    return int(servo_closed - normalized * (servo_closed - servo_open))

cap = cv2.VideoCapture(0)
previous = np.array([90, 90, 90, 90, 90], dtype=float)
ALPHA = 0.25

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        continue

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    result = landmarker.detect_for_video(mp_image, int(time.time() * 1000))

    if result.hand_landmarks:
        lm = result.hand_landmarks[0]
        values = [
```

```
        calc_angle(lm[2], lm[3], lm[4]),
        calc_angle(lm[5], lm[6], lm[8]),
        calc_angle(lm[9], lm[10], lm[12]),
        calc_angle(lm[13], lm[14], lm[16]),
        calc_angle(lm[17], lm[18], lm[20])
    ]
    current = np.array([angle_to_servo(x) for x in values], dtype=float)
    previous = ALPHA * current + (1 - ALPHA) * previous
    output = previous.astype(int)
    ser.write(",".join(map(str, output)) + "\n").encode())

cv2.imshow("Mao Robotica", frame)
if cv2.waitKey(1) & 0xFF == 27:
    break

cap.release()
ser.close()
landmarker.close()
cv2.destroyAllWindows()
```

APÊNDICE B - FIRMWARE ESP32 BASE

Este firmware recebe cinco posições CSV e as converte em pulsos para o PCA9685. Antes da montagem mecânica, reduza as faixas e faça testes individuais.

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);
const int SERVO_COUNT = 5;
int servoChannel[SERVO_COUNT] = {0, 1, 2, 3, 4};
int minPulseUs[SERVO_COUNT] = {700, 700, 700, 700, 700};
int maxPulseUs[SERVO_COUNT] = {2300, 2300, 2300, 2300, 2300};
bool inverted[SERVO_COUNT] = {false, false, false, false, false};

int microsecondsToTicks(int us) {
    return (int)((float)us / 20000.0) * 4096.0;
}

void setServoAngle(int servo, int angle) {
    angle = constrain(angle, 0, 180);
    if (inverted[servo]) angle = 180 - angle;
    int pulseUs = map(angle, 0, 180, minPulseUs[servo], maxPulseUs[servo]);
    pwm.setPWM(servoChannel[servo], 0, microsecondsToTicks(pulseUs));
}

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);
    pwm.begin();
    pwm.setPWMFreq(50);
    delay(500);
    for (int i = 0; i < SERVO_COUNT; i++) setServoAngle(i, 90);
}

void loop() {
    if (Serial.available()) {
        String line = Serial.readStringUntil('\n');
        int values[5];
        int read = sscanf(line.c_str(), "%d,%d,%d,%d,%d",
            &values[0], &values[1], &values[2], &values[3], &values[4]);

        if (read == 5) {
            for (int i = 0; i < 5; i++) setServoAngle(i, values[i]);
        }
    }
}
```

Próxima melhoria de firmware substituir os limites genéricos 0-180° por limites calibrados de cada dedo, adicionar timeout de comunicação e uma função de parada de emergência.

CONCLUSÃO

A construção recomendada transforma o conceito apresentado no vídeo em um processo de engenharia reproduzível: validar cada subsistema separadamente, estabelecer limites seguros e só depois integrar visão computacional, controle e mecânica. Essa metodologia reduz riscos, facilita o diagnóstico e cria uma base sólida para futuras melhorias com sensores, telemetria e controle adaptativo.

PROJETO • ROBÓTICA • VISÃO COMPUTACIONAL • IoT



UniFAJ - Centro Universitário de Jaguariúna